

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



BÁO CÁO MÔN HỌC
Các vấn đề hiện đại của Công nghệ thông tin

**HybridRAG: Integrating Knowledge Graphs and
Vector Retrieval Augmented Generation for Efficient
Information Extraction**

Giảng viên hướng dẫn	: TS. Nguyễn Đức Anh
Mã học phần	: INT3507 1
Sinh viên thực hiện	: Vũ Minh Khải - 22022556 Hồ Cảnh Quyền - 22022629 Nguyễn Đức Thành - 22022581 Bùi Tùng Lâm - 22021225

Mục lục

Mục lục	1
Tóm tắt	2
1 Tổng Quan	4
1.1 Vấn đề thực tế cần giải quyết	4
1.2 Giới thiệu các hướng RAG hiện nay	4
1.2.1 VectorRAG (Tìm kiếm theo nghĩa)	4
1.2.2 GraphRAG (Tìm kiếm theo quan hệ)	5
1.3 Bộ dữ liệu và lý do lựa chọn bài toán	5
2 Cơ sở lý thuyết	6
2.1 RAG	6
2.1.1 Truy xuất thông tin (Retriever)	6
2.1.2 Sinh câu trả lời (Generator)	6
2.2 Vector Retrieval	6
2.3 Graph Retrieval	7
2.4 HybridRAG	7
3 Quy trình triển khai	9
3.1 Thu thập dữ liệu	9
3.2 Xử lý dữ liệu	9
3.3 Xây dựng vector database	10
3.4 Xây dựng Knowledge Graph	10
3.5 Truy xuất dữ liệu từ graph	11
4 Cải tiến	12
4.1 Các cải tiến áp dụng cho VectorRAG	12
4.1.1 Triển khai hybrid search	12
4.1.2 Tích hợp rerank	12
4.2 Các cải tiến áp dụng cho GraphRAG	13

5	Thí nghiệm và đánh giá	14
5.1	Các loại truy vấn trong RAG	14
5.1.1	Single-Hop Query	14
5.1.2	Multi-Hop Query	14
5.1.3	Specific vs. Abstract Queries trong RAG	15
5.2	Bộ dữ liệu kiểm thử	16
5.3	Các metric đánh giá.	16
5.3.1	Faithfulness	16
5.3.2	Answer Relevancy (Độ liên quan của câu trả lời)	17
5.3.3	Context Precision (Độ chính xác ngữ cảnh)	19
5.3.4	Context Recall (Độ phủ ngữ cảnh)	19
5.4	Đánh giá	20
5.4.1	Quy trình tham chiếu đánh giá.	20
5.4.2	Cấu hình chi tiết các thành phần trong hệ thống	20
5.4.3	Kết quả đánh giá	20
6	Kết luận, hạn chế và hướng phát triển	22
6.1	Kết luận	22
6.2	Hạn chế của hệ thống	22
6.3	Hướng nghiên cứu mở rộng	23
7	Hệ thống hỏi đáp	24
7.1	Một số hình ảnh chương trình	24
7.2	Ví dụ câu hỏi – câu trả lời	26
7.2.1	Ví dụ 1: Truy vấn gợi ý sản phẩm	26
7.2.2	Ví dụ 2: So sánh sản phẩm	26
7.2.3	Ví dụ 3: Cung cấp thông tin	27
	Tài liệu tham khảo	29

Tóm tắt

Trong báo cáo này, nhóm trình bày quá trình triển khai một hệ thống tư vấn khách hàng cho website bán điện thoại dựa trên kiến trúc Hybrid Retrieval-Augmented Generation (HybridRAG - kết hợp giữa VectorRAG và GraphRAG). VectorRAG được sử dụng để truy vấn các tài liệu liên quan dựa trên độ tương đồng ngữ nghĩa giữa truy vấn người dùng và embedding của dữ liệu. GraphRAG được sử dụng để tìm kiếm đồ thị bao gồm các thực thể và mối quan hệ như mẫu điện thoại, hãng sản xuất, thông số kỹ thuật và giá bán,... để cung cấp ngữ cảnh bổ sung thông tin. Hai cơ chế truy xuất này giúp cung cấp ngữ cảnh đầy đủ và chính xác hơn cho mô hình ngôn ngữ trong quá trình sinh câu trả lời. Kết quả đánh giá cho thấy hệ thống Hybrid RAG cải thiện đáng kể độ chính xác, mức độ tin cậy của câu trả lời so với các hệ thống chỉ sử dụng VectorRAG. Báo cáo cũng cho thấy rằng HybridRAG có thể triển khai trong thực tế cho bài toán tư vấn khách hàng trong thương mại điện tử và có tiềm năng ứng dụng sang các bài toán thực tế khác...

Chương 1

Tổng Quan

1.1 Vấn đề thực tế cần giải quyết

Các mô hình ngôn ngữ lớn (LLM) đã đạt được nhiều tiến bộ vượt bậc trong việc hiểu và sinh ngôn ngữ tự nhiên. Tuy nhiên, LLM vẫn còn nhiều hạn chế khi được áp dụng vào các bài toán hỏi đáp cần thông tin chính xác...

- Tạo ra thông tin sai gây ra “ảo giác” (hallucination): Đôi khi trả lời những điều không có trong tài liệu.
- Giới hạn về độ dài ngữ cảnh (Context Window): Mỗi LLM chỉ có thể xử lý một lượng từ (token) giới hạn trong một lần. Các tài liệu dài sẽ vượt xa giới hạn LLM không thể nắm bắt toàn bộ ngữ cảnh của tài liệu để đưa ra câu trả lời toàn diện và chính xác nhất.
- LLM không thể trả lời các câu hỏi thuộc miền dữ liệu mà nó chưa được học. Cần một kĩ thuật để giải quyết vấn đề này.

1.2 Giới thiệu các hướng RAG hiện nay

Để giải quyết vấn đề trên, kĩ thuật phổ biến nhất hiện nay là RAG (Retrieval-Augmented Generation), bổ sung thông tin tham khảo cho LLM. Chúng ta có thể chia RAG thành 2 hướng chính: VectorRAG và GraphRAG.

1.2.1 VectorRAG (Tìm kiếm theo nghĩa)

VectorRAG: tài liệu và truy vấn được biểu diễn dưới dạng vector embedding, truy xuất ngữ cảnh dựa trên độ tương đồng ngữ nghĩa giữa truy vấn và dữ liệu. Hiệu quả nhờ khả năng tìm kiếm theo ngữ nghĩa. Có hạn chế bị mất ngữ cảnh do chunking, kém với câu hỏi trích xuất thông tin.

1.2.2 GraphRAG (Tìm kiếm theo quan hệ)

GraphRAG: sử dụng đồ thị tri thức để truy xuất ngữ cảnh từ đồ thị bao gồm các thực thể và mối quan hệ của chúng. Rất mạnh khi sử dụng cho các câu hỏi cần trích xuất thông tin. Hạn chế với các câu hỏi trừu tượng,...

1.3 Bộ dữ liệu và lý do lựa chọn bài toán

Nhóm xây dựng hệ thống tư vấn khách hàng cho website bán điện thoại. Do đó, để xây dựng bộ dữ liệu sát thực tế nhất, nhóm đã thu thập dữ liệu từ CellphoneS bằng thư viện playwright python. Bộ dữ liệu thu thập được bao gồm các thông tin liên quan đến sản phẩm điện thoại như tên sản phẩm, cấu hình kỹ thuật, giá bán, các mô tả chi tiết,...

Lý do nhóm chọn chủ đề hệ thống tư vấn khách hàng cho website bán điện thoại xuất phát từ yêu cầu của dự án, trong đó hệ thống chỉ được phép sử dụng các mô hình ngôn ngữ có quy mô nhỏ, khoảng 1 tỷ tham số. Các mô hình này có ưu điểm là chi phí triển khai thấp, thời gian suy luận nhanh và phù hợp với các hệ thống có tài nguyên hạn chế, tuy nhiên lại gặp hạn chế về khả năng ghi nhớ kiến thức và suy luận phức tạp. Chủ đề này đủ nhỏ để mô hình ngôn ngữ nhỏ hoạt động hiệu quả.

Chương 2

Cơ sở lý thuyết

2.1 RAG

Định nghĩa: Retrieval-Augmented Generation (RAG) là mô hình kết hợp hai giai đoạn: Truy xuất thông tin (retrieval) và sinh câu trả lời (generation). RAG lấy kiến thức từ cơ sở dữ liệu bên ngoài, cung cấp thông tin giúp mô hình ngôn ngữ lớn tạo ra câu trả lời chính xác. Cách tiếp cận này giải quyết hai vấn đề lớn mà LLM thường gặp phải, đó là hiện tượng ảo giác, mô hình tự tưởng tượng thông tin không có thật và vấn đề mô hình không trả lời được các câu hỏi về miền thông tin nó chưa được học.

2.1.1 Truy xuất thông tin (Retriever)

Retriever thực hiện việc tìm kiếm các đoạn văn bản liên quan từ cơ sở dữ liệu. Các bước thực hiện:

- Câu hỏi của người dùng được mã hoá thành vector bằng mô hình embedding
- Hệ thống so sánh vector truy vấn với các vector đã được lưu trong cơ sở dữ liệu để tìm ra các tài liệu có mức tương đồng cao nhất..
- Lọc lấy top-k tài liệu liên quan nhất.

2.1.2 Sinh câu trả lời (Generator)

Sau khi Retriever truy xuất được top-k tài liệu liên quan nhất đến câu hỏi người dùng, những tài liệu này được gửi kèm với câu hỏi người dùng làm thông tin bổ sung để LLM căn cứ vào đó trả lời. Do có thông tin bổ sung đi kèm, LLM căn cứ vào đó nên giảm thiểu được hiện tượng ảo giác, đồng thời nâng cao độ chính xác của câu trả lời so với việc chỉ sử dụng LLM để tạo câu trả lời.

2.2 Vector Retrieval

Đối với Vector Retrieval, quá trình này sẽ bắt đầu bằng việc bắt đầu với 1 câu hỏi truy vấn. Câu hỏi này được sử dụng để tìm kiếm các thông tin liên quan có trong cơ sở

dữ liệu.

Các đoạn văn bản đã được truy xuất sau đó được sử dụng làm ngữ cảnh đầu vào gửi cho mô hình. Điều này có thể giúp cho các câu trả lời của mô hình có tính xác thực, làm tăng độ chính xác và độ liên quan giữa câu hỏi và câu trả lời.

Do các mô hình ngôn ngữ bị giới hạn bởi độ dài context, nên trước khi đưa các mẫu văn bản này vào mô hình thì văn bản sẽ được chia thành nhiều đoạn văn bản nhỏ hơn hay còn được gọi là chunks. Các chunks dữ liệu này sẽ được embedding và lưu lại trong 1 hệ thống vector database.

Sau đó, hệ thống Vector Retrieval sẽ tính toán mức độ tương đồng giữa query và các chunks đã được embed trong vector database để từ đó lấy ra được các đoạn chunks có xếp hạng cao nhất để nối lại thành context cho mô hình ngôn ngữ.

2.3 Graph Retrieval

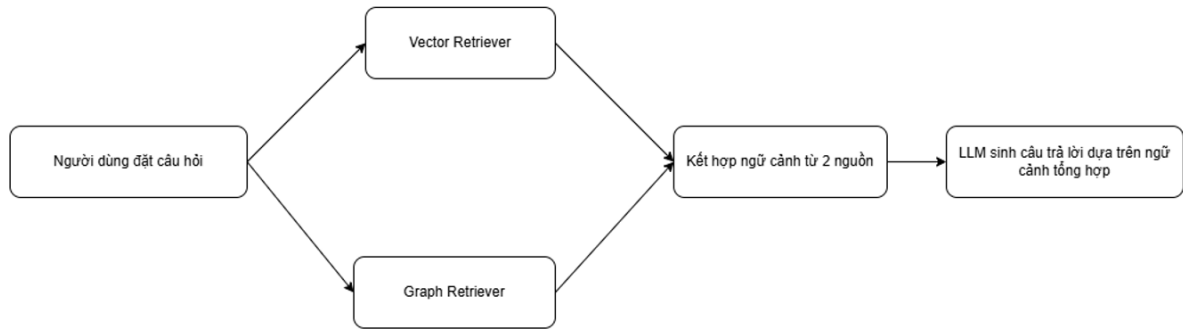
Graph Retrieval, bắt đầu với việc xây dựng 1 đồ thị tri thức có hướng (Knowledge Graph - KG), trong đó các nút đại diện cho một thực thể, còn các cạnh đại diện cho quan hệ có nghĩa giữa các thực thể đó. Các thực thể có thể là "con người", sự kiện, sản phẩm,... còn các quan hệ là sự kết nối giữa các thực thể như: "là của", "sử dụng bởi", "có",... GraphRAG có nhiều phiên bản, phiên bản đơn giản thực hiện như sau: Với mỗi câu hỏi người dùng, bước đầu tiên là trích xuất các thực thể từ câu hỏi người dùng. Từ các thực thể này, tìm ra các nút tương ứng với chúng, rồi từ các nút này, lấy kết quả theo một độ sâu k từ đồ thị. Kết quả sau đó được gửi cho LLM cùng với câu hỏi người dùng ban đầu để sinh câu trả lời.

2.4 HybridRAG

Ý tưởng: HybridRAG^[1] là kết hợp đồng thời hai cơ chế truy xuất: Vector Retrieval và Graph Retrieval nhằm tận dụng ưu điểm của 2 phương pháp này. Vector Retrieval cho phép tìm kiếm thông tin dựa vào độ tương đồng ngữ nghĩa giữa câu hỏi và tài liệu. Graph Retrieval có khả năng liên kết quan hệ giữa các thực thể trong đồ thị tri thức, là nguồn dữ liệu bổ sung

Triển khai:

- Người dùng đặt câu hỏi
- Vector Retriever: Truy xuất các tài liệu gần về ngữ nghĩa với câu hỏi.



Hình 2.1: HybridRAG

- Graph Retriever: Truy xuất các thực thể và quan hệ liên quan trong câu hỏi.
- Kết hợp kết quả từ hai nguồn
- LLM sinh câu trả lời cuối cùng dựa trên ngữ cảnh tổng hợp.

Sau khi truy xuất được các tài liệu liên quan từ Vector Retriever và Graph Retriever, hệ thống sắp xếp các tài liệu từ Vector Retriever lên trước, sau đó là các tài liệu từ Graph Retriever, rồi ghép chúng vào prompt có sẵn để tạo ngữ cảnh đầu vào cho LLM sinh câu trả lời.

Chương 3

Quy trình triển khai

3.1 Thu thập dữ liệu

Quá trình thu thập dữ liệu bắt đầu như sau:

- Bước 1: Thực hiện quét các url của danh sách tất cả các điện thoại trong item list của website gốc.
- Bước 2: Thực hiện việc cào lấy dữ liệu thông qua danh sách các url của bước trước.
- Bước 3: Triển khai việc lưu trữ các dữ liệu mới vào trong database Postgresql.

Thông tin của dữ liệu được cào bao gồm có các thông tin như sau:

- title: Tên của sản phẩm
- price: Giá thành của sản phẩm
- colors: Các phiên bản màu sắc của sản phẩm
- specs: Các thông số kỹ thuật của sản phẩm đó
- images: Ảnh của sản phẩm
- description: Mô tả sản phẩm

3.2 Xử lý dữ liệu

Dữ liệu sau khi đã được lưu trữ sẽ được lấy ra dưới dạng markdown sao cho dữ liệu đó thuận tiện cho việc truy xuất thông tin về sau. Cụ thể, dữ liệu sẽ được lưu theo định dạng như ví dụ sau:

```
1 {  
2   "title": "iPhone 15 Pro Max",  
3   "price": "34.990.000 VND",  
4   "colors": [  
5     "Titan Tu nhien",
```

```

6     "Titan Xanh",
7     "Titan Trang",
8     "Titan Đen"
9 ],
10 "specs": {
11     "screen": "6.7 inch",
12     "chip": "A17 Pro",
13     "ram": "8GB"
14 },
15 "images": [
16     "https://example.com/img1.jpg",
17     "https://example.com/img2.jpg"
18 ],
19 "description": "iPhone 15 Pro Max là điện thoại cao cấp nhất..."
20 }

```

3.3 Xây dựng vector database

Các file markdown dữ liệu được chia thành nhiều chunks khác nhau. Các chunks này được embedding theo kiểu hybrid search tức là kết hợp cả 2 phương pháp dense embedding và sparse embedding. Điều này giúp tận dụng được lợi thế lexical matching của sparse embedding đồng thời tận dụng được lợi thế semantic matching của dense embedding. Các mô hình được sử dụng cho quá trình embedding bao gồm:

- Dense: BAAI/bge-m3
- Sparse: Qdrant/bm25

3.4 Xây dựng Knowledge Graph

Từ các file dữ liệu ban đầu, áp dụng các bước triển khai sau để bóc tách thực thể và quan hệ sau đó lưu vào cơ sở dữ liệu đồ thị.

- Chia dữ liệu thành các chunk với (`chunk_size = 512`, `overlap = 126`)
- Với mỗi chunk, dùng LLM để tóm tắt nội dung nhằm làm rõ ngữ cảnh chính.
- Trích xuất thực thể và quan hệ bằng LLM
- Mỗi quan hệ được biểu diễn theo dạng bộ ba (`subject`, `relation`, `object`).

- Lưu vào cơ sở dữ liệu đồ thị (NetworkX) để hỗ trợ truy vấn theo liên kết thực thể.
- Tạo embedding cho các node trong graph và lưu vào vector database.

3.5 Truy xuất dữ liệu từ graph

Từ câu hỏi người dùng, áp dụng các bước sau để truy xuất thông tin:

- Trích xuất thực thể liên quan từ câu hỏi người dùng.
- Với mỗi thực thể, sử dụng tìm kiếm vector và tìm kiếm theo cú pháp để xác định các node phù hợp.
- Truy vấn các bộ ba liên quan từ graph từ các node, sử dụng độ sâu bằng 1.

Chương 4

Cải tiến

Nhóm áp dụng một số cải tiến để tăng độ chính xác, đặc biệt là với trường hợp mô hình LLM sử dụng chỉ có 1B tham số. Vì HybridRAG là sự kết hợp của VectorRAG và GraphRAG, các cải tiến có thể chia thành 2 nhóm, dành cho VectorRAG và GraphRAG

4.1 Các cải tiến áp dụng cho VectorRAG

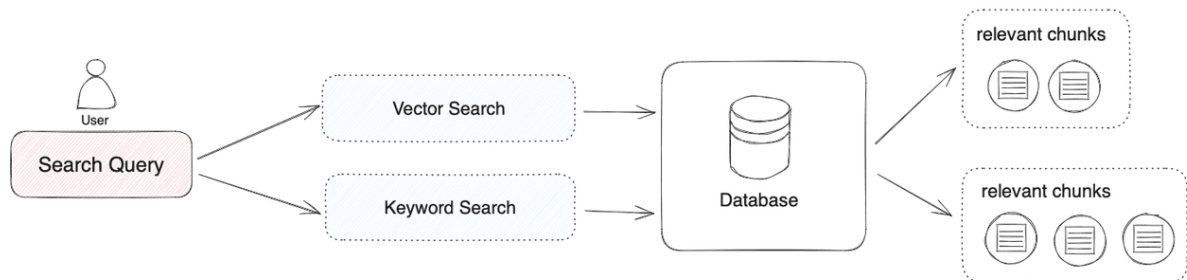
4.1.1 Triển khai hybrid search

Phần tìm kiếm tài liệu, thay vì chỉ sử dụng vector search như VectorRAG thông thường, bổ sung thêm 1 phần tìm kiếm keyword search song song để khớp từ khóa. Các kết quả được gộp lại và tính điểm bằng công thức trọng số kết hợp giữa điểm dense và điểm sparse. Điểm tổng hợp này được dùng để sắp xếp và chọn ra các tài liệu phù hợp nhất.

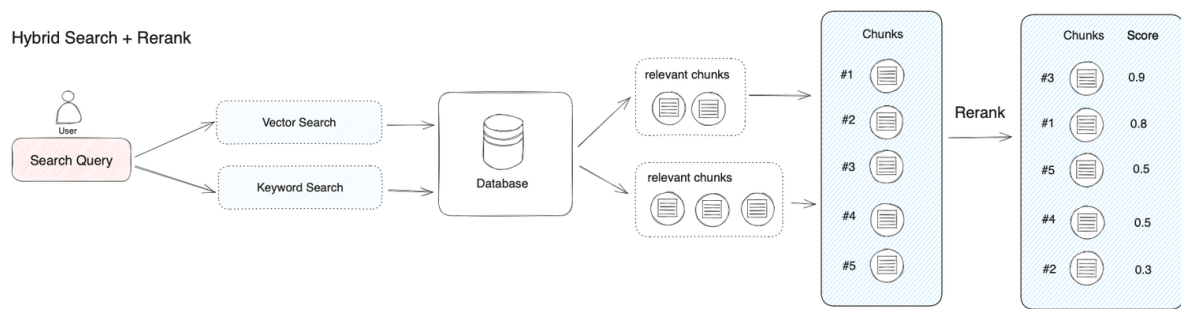
4.1.2 Tích hợp rerank

Sau khi truy vấn được các tài liệu từ VectorRAG, sử dụng một mô hình cross-encoder để đánh giá lại độ liên quan giữa truy vấn và các tài liệu. Từng cặp tài liệu và câu hỏi được sử dụng làm đầu vào cho mô hình cross encoder để tính điểm relevance. Dựa trên các điểm số này, các tài liệu được sắp xếp lại theo thứ tự phù hợp.

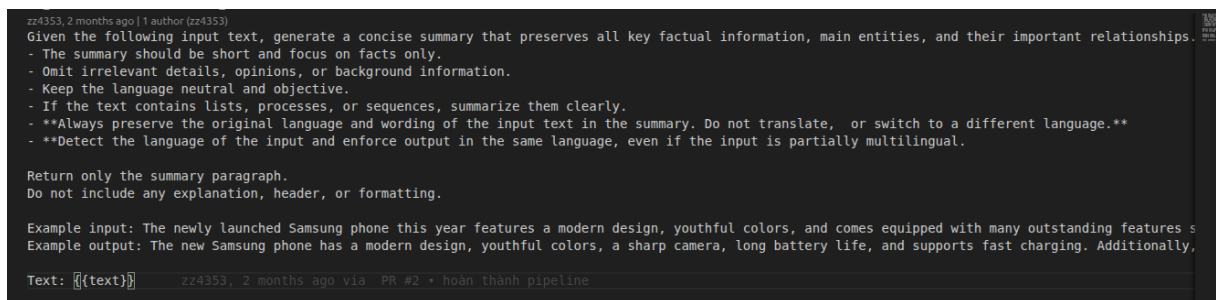
Hybrid Search



Hình 4.1: Hybrid search



Hình 4.2: Rerank



Hình 4.3: Prompt tóm tắt dữ liệu

4.2 Các cải tiến áp dụng cho GraphRAG

Bổ sung một lớp tìm kiếm node liên quan sau khi trích xuất thực thể từ câu hỏi. Các node được chọn là kết quả kết hợp giữa hai phương pháp sau:

- Embedding các node của graph, sau đó sử dụng vector search để tìm kiếm các node có độ tương đồng ngữ nghĩa với các thực thể từ câu hỏi.
- Sử dụng LSH index để tìm các node có mức độ tương đồng về mặt ký tự với thực thể được trích xuất từ câu hỏi.

Ở bước tóm tắt nội dung các chunk dữ liệu, prompt yêu cầu LLM:

- Khi văn bản chứa các đại từ chỉ các thực thể được đặt tên có thể xác định được, thay thế các đại từ đó bằng danh từ riêng ban đầu.
- Lược bỏ các chi tiết, ý kiến hoặc thông tin cơ bản không liên quan.

Chương 5

Thí nghiệm và đánh giá

Giới thiệu RAGAS: thư viện giúp đánh giá có hệ thống vòng lặp cho AI applications. Nó cung cấp tools để đánh giá các ứng dụng của mô hình ngôn ngữ lớn (LLM). Vậy tại sao chúng ta sử dụng RAGAS? Các metric đánh giá truyền thống không thể nắm bắt được những vấn đề quan trọng của LLM applications. Đánh giá thủ công không thể mở rộng. RAGAS giải quyết vấn đề này bằng cách kết hợp LLM-driven metric với thử nghiệm có hệ thống để tạo một vòng lặp cải tiến liên tục.

5.1 Các loại truy vấn trong RAG

Để phát triển một bộ dữ liệu đánh giá tốt trước hết chúng ta sẽ đi vào tìm hiểu một số loại truy vấn chủ yếu cho một hệ thống rag system.

5.1.1 Single-Hop Query

Truy vấn một bước (single-hop query) là một câu hỏi đơn giản, yêu cầu truy xuất thông tin từ một tài liệu hoặc nguồn duy nhất để đưa ra câu trả lời phù hợp. Nó chỉ bao gồm một bước duy nhất để có được câu trả lời.

Ví dụ (Truy vấn cụ thể):

“Điện thoại Iphone17 ra đời vào năm nào?” Đây là một câu hỏi cụ thể, dựa trên thực tế và có thể được trả lời chỉ bằng một lần tra cứu từ tài liệu chứa thông tin đó.

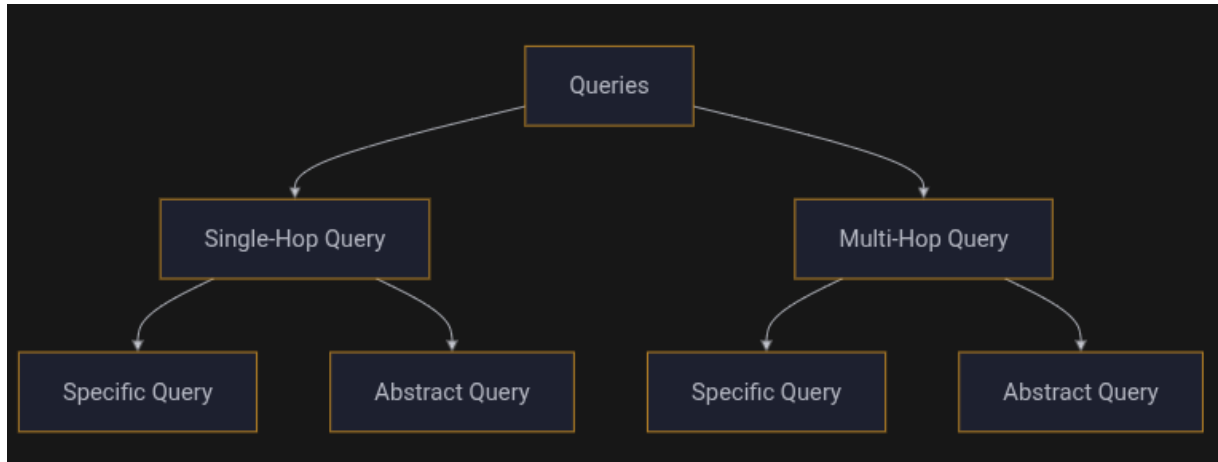
Ví dụ (Truy vấn trừu tượng):

“Iphone17 có ý nghĩa gì đối với công ty Apple Inc không?” Mặc dù câu hỏi này vẫn đề cập đến một khái niệm duy nhất, nhưng nó đòi hỏi một lời giải thích trừu tượng hơn hoặc mang tính diễn giải từ tài liệu nguồn.

5.1.2 Multi-Hop Query

Multi-Hop Query là dạng câu hỏi truy vấn cần phải lấy dữ liệu từ nhiều nguồn. Qua việc truy xuất thông tin từ nhiều tài liệu khác nhau và kết nối các điểm thông tin để tạo ra câu trả lời chính xác.

Ví dụ (Truy vấn cụ thể):



Hình 5.1: Minh họa các loại truy vấn

“Apple Inc là tập đoàn công nghệ đa quốc gia của Mỹ chuyên thuyết kế và phát triển gì, và tập đoàn này được định giá bao nhiêu?” Điều này đòi hỏi system phải thu thập thông tin về định hướng phát triển của tập đoàn và thông tin định giá của tập đoàn. Do đó hệ thống cần lấy dữ liệu hai nguồn khác nhau.

Ví dụ (Truy vấn trừu tượng):

“Công ty Apple Inc đã phát triển như thế nào trong giai đoạn mới thành lập đến hiện tại và tương lai sẽ có định hướng như thế nào?”

Câu hỏi trừu tượng này đòi hỏi việc thu thập nhiều thông tin theo thời gian và từ nhiều nguồn khác nhau để đưa ra một câu trả lời mang tính trừu tượng và bao quát.

5.1.3 Specific vs. Abstract Queries trong RAG

Truy vấn cụ thể: Tập trung vào việc truy xuất thông tin rõ ràng, dựa trên thực tế. Mục tiêu trong RAG là truy xuất thông tin có liên quan cao từ một hoặc nhiều tài liệu trực tiếp giải đáp câu hỏi cụ thể.

Truy vấn trừu tượng: Yêu cầu phản hồi rộng hơn, mang tính diễn giải cao hơn. Trong RAG, các truy vấn trừu tượng thách thức hệ thống truy xuất phải lấy thông tin từ các tài liệu chứa lập luận, giải thích hoặc ý kiến ở cấp độ cao hơn, thay vì chỉ là các sự kiện đơn giản.

Trong cả trường hợp truy vấn một bước và nhiều bước, sự phân biệt giữa truy vấn cụ thể và truy vấn trừu tượng định hình quá trình truy xuất và tạo truy vấn bằng cách xác định xem trọng tâm là độ chính xác (cụ thể) hay là tổng hợp các ý tưởng rộng hơn (trừu tượng).

```

{
  "question": "Tại sao điện thoại Viettel T2 4G lại có thời gian sử dụng lâu dài?",
  "answer": "Vì nó được trang bị viên pin 1400mAh."
},
{
  "question": "Camera sau của Viettel T2 4G có độ phân giải là bao nhiêu?",
  "answer": "0.08MP"
},
{
  "question": "Kích thước màn hình của Viettel T2 4G là bao nhiêu inch?",
  "answer": "2.4 inches"
}

```

Hình 5.2: data demo

Các loại truy vấn khác nhau yêu cầu tổng hợp các ngữ cảnh khác nhau. Để giải quyết vấn đề này, Ragas sử dụng phương pháp dựa trên Đồ thị Tri thức để tạo tập dữ liệu kiểm thử.

5.2 Bộ dữ liệu kiểm thử

Bộ kiểm thử của chúng tôi được generate dựa trên cấu trúc các loại truy vấn được đề cập trong phần trước. Mỗi file dữ liệu markdown được chúng tôi xử lý bằng LLM để sinh 3 question và ground truth tương ứng. Tổng số lượng câu hỏi là 300 câu.

5.3 Các metric đánh giá.

5.3.1 Faithfulness

Chỉ số Faithfulness dùng để đánh giá mức độ phù hợp và chính xác của câu trả lời so với thông tin tham khảo đã được truy xuất. Chỉ số này nằm trong khoảng từ 0 đến 1, với điểm số càng cao thì câu trả lời càng bám sát so với dữ liệu ngữ cảnh được truy xuất. Một câu trả lời trung thực nếu tất cả các luận điểm của nó đều có thể tìm thấy hoặc suy ra từ ngữ cảnh. Để tính toán sử dụng các bước:

1. Xác định tất cả các luận điểm trong câu trả lời.
2. Kiểm tra từng luận điểm để xem liệu nó có thể được suy luận từ ngữ cảnh đã thu thập được hay không.
3. Tính điểm độ trung thực bằng công thức:

$$\text{Faithfulness} = \frac{|\text{Number of claims in generated answer inferable from context}|}{|\text{Total number of claims in generated answer}|} \quad (5.1)$$

Example

Question: Einstein sinh ra ở đâu và khi nào?

Context: Albert Einstein (sinh ngày 14 tháng 3 năm 1879) là một nhà vật lý lý thuyết người Đức, được nhiều người coi là một trong những nhà khoa học vĩ đại và có ảnh hưởng nhất mọi thời đại.

High faithfulness answer: Einstein sinh ra ở Đức vào ngày 14 tháng 3 năm 1879.

Low faithfulness answer: Einstein sinh ra ở Đức vào ngày 20 tháng 3 năm 1879.

Hãy cùng xem xét cách tính toán Faithfulness dựa trên câu trả lời có mức độ trung thực thấp:

Bước 1: Chia câu trả lời được tạo ra thành các mệnh đề riêng lẻ.

- Mệnh đề 1: “Einstein sinh ra ở Đức.”
- Mệnh đề 2: “Einstein sinh ngày 20 tháng 3 năm 1879.”

Bước 2: Đối với mỗi mệnh đề phát biểu được tạo ra, hãy xác minh xem nó có thể được suy luận từ ngữ cảnh đã cho hay không.

- Mệnh đề 1: Có
- Mệnh đề 2: Không

Bước 3: Sử dụng công thức được nêu ở trên để tính toán Faithfulness.

$$\text{Faithfulness} = \frac{1}{2} = 0.5 \quad (5.2)$$

5.3.2 Answer Relevancy (Độ liên quan của câu trả lời)

Chỉ số Độ phù hợp của câu trả lời đo lường mức độ phù hợp của câu trả lời với thông tin người dùng nhập vào. Chỉ số này nằm trong khoảng từ 0 đến 1, với điểm số càng cao cho thấy sự phù hợp càng tốt với thông tin người dùng nhập vào.

Câu trả lời được coi là phù hợp nếu nó trực tiếp và đáp ứng đúng câu hỏi ban đầu. Tiêu chí này tập trung vào mức độ phù hợp của câu trả lời với ý định của câu hỏi, mà không đánh giá tính chính xác về mặt thực tế. Nó sẽ trừ điểm những câu trả lời không đầy đủ hoặc bao gồm các chi tiết không cần thiết.

Chỉ số này được tính toán bằng cách sử dụng *user_input* và *response* như sau:

1. Tạo một bộ câu hỏi giả định (mặc định là 3) dựa trên câu trả lời. Những câu hỏi này được thiết kế để phản ánh nội dung của câu trả lời.

2. Tính toán độ tương đồng cosin giữa phần nhúng của dữ liệu đầu vào của người dùng (E_o) và sự nhúng của mỗi câu hỏi được tạo ra (E_{g_i}).
3. Lấy giá trị trung bình của các điểm tương đồng cosine này để có được Độ phù hợp của câu trả lời.

$$\text{Answer Relevancy} = \frac{1}{N} \sum_{i=1}^N \text{cosine similarity}(E_{g_i}, E_o) \quad (5.3)$$

Công thức trên có thể được viết tương đương dưới dạng:

$$\text{Answer Relevancy} = \frac{1}{N} \sum_{i=1}^N \frac{E_{g_i} \cdot E_o}{\|E_{g_i}\| \|E_o\|} \quad (5.4)$$

Trong đó:

- E_{g_i} : vector embedding của câu hỏi được sinh ra thứ i .
- E_o : vector embedding của câu hỏi đầu vào của người dùng.
- N : số lượng câu hỏi được sinh ra (mặc định là 3, có thể cấu hình thông qua tham số `strictness`).

Lưu ý: Mặc dù điểm số thường nằm trong khoảng từ 0 đến 1, nhưng điều này không được đảm bảo do miền giá trị toán học của độ tương đồng cosine nằm trong khoảng $[-1, 1]$.

Ví dụ

Question: Nước Pháp nằm ở đâu và thủ đô của nó là gì?

Low relevance answer: Pháp nằm ở Tây Âu.

High relevance answer: Pháp nằm ở Tây Âu và Paris là thủ đô của nước này.

Để đánh giá mức độ phù hợp của câu trả lời với câu hỏi đã cho, chúng ta thực hiện theo hai bước:

Bước 1: Phân tích ngược n biến thể của câu hỏi từ câu trả lời được tạo ra bằng cách sử dụng Mô hình Ngôn ngữ Lớn (LLM). Ví dụ, với câu trả lời đầu tiên, LLM có thể tạo ra các câu hỏi khả thi sau:

- Câu hỏi 1: “Nước Pháp nằm ở khu vực nào của châu Âu?”
- Câu hỏi 2: “Vị trí địa lý của Pháp trong châu Âu là ở đâu?”

- Câu hỏi 3: “Bạn có thể xác định khu vực nào của châu Âu mà nước Pháp tọa lạc không?”

Bước 2: Tính độ tương đồng cosin trung bình giữa câu hỏi được tạo ra và câu hỏi thực tế.

Khái niệm cơ bản là nếu câu trả lời giải đáp chính xác câu hỏi, thì rất có thể câu hỏi ban đầu có thể được khôi phục lại chỉ từ câu trả lời đó.

5.3.3 Context Precision (Độ chính xác ngữ cảnh)

Độ chính xác ngữ cảnh là một chỉ số đánh giá khả năng của trình truy xuất trong việc xếp hạng các đoạn thông tin liên quan cao hơn các đoạn thông tin không liên quan cho một truy vấn nhất định trong ngữ cảnh được truy xuất. Cụ thể, nó đánh giá mức độ mà các đoạn thông tin liên quan trong ngữ cảnh được truy xuất được đặt ở vị trí đầu bảng xếp hạng.

Nó được tính bằng giá trị trung bình của precision@k cho mỗi khối trong ngữ cảnh. Precision@k là tỷ lệ giữa số lượng khối liên quan ở thứ hạng k với tổng số khối ở thứ hạng k .

$$\text{Context Precision@K} = \frac{\sum_{k=1}^K (\text{Precision@k} \times v_k)}{\text{Total number of relevant items in the top } K \text{ results}} \quad (5.5)$$

$$\text{Precision@k} = \frac{\text{true positives@k}}{\text{true positives@k} + \text{false positives@k}} \quad (5.6)$$

Ở đây K là tổng số khối trong `retrieved_contexts` và $v_k \in \{0, 1\}$ là chỉ số mức độ liên quan ở cấp bậc k .

5.3.4 Context Recall (Độ phủ ngữ cảnh)

Chỉ số Context Recall đo lường số lượng tài liệu (hoặc thông tin) liên quan được truy xuất thành công. Nó tập trung vào việc không bỏ sót các kết quả quan trọng. Chỉ số Context Recall cao hơn có nghĩa là ít tài liệu liên quan bị bỏ sót hơn. Tóm lại, Context Recall là về việc không bỏ sót bất kỳ thông tin quan trọng nào.

Vì mục tiêu là không bỏ sót bất cứ điều gì, việc tính toán khả năng nhớ lại ngữ cảnh luôn yêu cầu một tham chiếu để so sánh. Chỉ số Khả năng nhớ lại ngữ cảnh dựa trên LLM sử dụng reference làm đại diện cho *reference_contexts*, điều này giúp việc sử dụng dễ dàng hơn vì việc chú thích ngữ cảnh tham chiếu có thể rất tốn thời gian. Để ước tính

khả năng nhớ lại ngữ cảnh từ reference, tham chiếu được chia nhỏ thành các tuyên bố, và mỗi tuyên bố được phân tích để xác định xem nó có thể được gán cho ngữ cảnh được truy xuất hay không. Trong trường hợp lý tưởng, tất cả các tuyên bố trong câu trả lời tham chiếu đều phải được gán cho ngữ cảnh được truy xuất.

Công thức tính toán khả năng nhớ lại ngữ cảnh như sau:

$$\text{Context Recall} = \frac{|\text{GT claims attributed to context}|}{|\text{Number of claims in GT}|} \quad (5.7)$$

5.4 Đánh giá

5.4.1 Quy trình tham chiếu đánh giá.

1. User Question → 2. Retriever (Vector DB + Embedding) → 3. Retrieved Contexts → 4. LLM Generator → 5. Answer

- Các chỉ số Context Precision và Context Recall được tính toán dựa trên dữ liệu ở bước số 1, 2 và 3. Context Precision và Context Recall so sánh Retrieved Contexts (kết quả tìm kiếm) với Ground Truth (đáp án chuẩn), hoàn toàn chưa đụng đến bước số 4.
- Faithfulness : Câu trả lời có bịa đặt (hallucination) so với Context không? Công thức cốt lõi: So sánh (3) Retrieved Contexts ↔ (5) Answer
- Answer Relevancy (Mức độ liên quan) Câu hỏi: Câu trả lời có đúng trọng tâm câu hỏi không? Công thức cốt lõi: So sánh (1) User Question ↔ (5) Answer

5.4.2 Cấu hình chi tiết các thành phần trong hệ thống

xem bảng 5.1.

5.4.3 Kết quả đánh giá

Bảng kết quả đánh giá trên bộ test 300 câu. Nhờ áp dụng hybrid search và rerank, kết quả đánh giá VectorRAG và HybirRAG tăng mạnh, đặc biệt là kết quả HybridRAG do còn được áp dụng thêm kĩ thuật sử dụng LSH index để tìm node liên quan từ graph nên kết quả tăng mạnh hơn. Xem bảng 5.2: So sánh hiệu suất các phương pháp RAG trên mô hình gemma3:1b. Với VectorRAG và HybridRAG là cài đặt cơ bản, VectorRAG+ và HybridRAG+ là phiên bản đã áp dụng các cải tiến.

Bảng 5.1: Cấu hình chi tiết các thành phần trong hệ thống

Hạng mục	Thành phần	Giá trị	Ghi chú
1. RAG system	LLM chính	<code>gemma3:1b</code>	Chạy cục bộ (local)
	Vector Database	Qdrant	localhost
	Graph Database	NetworkxEntityGraph	Langchain
	Mô hình Embedding	BAAI/bge-m3	Mô hình đa ngôn ngữ
	Mô hình Reranker	namdp- <code>ptit</code> /ViRanker	Mô hình chuyên cho tiếng Việt
	Nguồn dữ liệu	Crawler	<code>cellphones.com.vn</code>
2. Test Set	LLM tạo dữ liệu	<code>gpt-4o-mini</code>	Sử dụng OpenAI API
	Temperature	0	Cho kết quả nhất quán
	Số lượng QAs / File	3	
	Giới hạn ký tự đầu vào	8000	Tránh prompt quá dài
3. Evaluate	Critic LLM (Ragas)	<code>gpt-4o-mini</code>	Sử dụng OpenAI API để "chấm điểm"
	Top K (khi đánh giá)	3	Số lượng ngữ cảnh truy xuất
	Các chỉ số (Metrics)	<code>faithfulness</code> , <code>answer_relevancy</code> , <code>context_recall</code> , <code>context_precision</code>	Đánh giá chất lượng câu trả lời và ngữ cảnh

Bảng 5.2: So sánh hiệu suất các phương pháp RAG trên mô hình `gemma3:1b`

Metric	VectorRAG	HybridRAG	VectorRAG+	HybridRAG+
Context Precision	0.76	0.80	0.80	0.82
Context Recall	0.71	0.74	0.78	0.80
Faithfulness	0.77	0.79	0.79	0.81
Answer Relevancy	0.68	0.71	0.79	0.81

Chương 6

Kết luận, hạn chế và hướng phát triển

6.1 Kết luận

Trong báo cáo này, nhóm đã thực hiện triển khai và cải tiến kỹ thuật HybridRAG sử dụng cho hệ thống tư vấn khách hàng website bán điện thoại. HybridRAG kế thừa ưu điểm của truy xuất theo ngữ nghĩa từ VectorRAG và truy xuất theo quan hệ từ đồ thị tri thức của GraphRAG để cung cấp ngữ cảnh đầy đủ hơn cho mô hình ngôn ngữ lớn trong quá trình sinh câu trả lời.

Kết quả thực nghiệm cho thấy HybridRAG cho chất lượng câu trả lời tốt hơn đáng kể so với cài đặt RAG truyền thống. Việc áp dụng các cải tiến hybrid search và rerank dành cho vectorRAG và cải tiến về prompt, sử dụng LSH Index để tìm kiếm nút cho GraphRAG giúp tăng đáng kể kết quả trên bộ đánh giá.

Việc áp dụng HybridRAG cho thấy hiệu quả vượt trội so với RAG truyền thống, hiện tại nhóm mới chỉ cài đặt và triển khai trên miền dữ liệu nhỏ, nhiệm vụ đơn giản, có tiềm năng để mở rộng sau này.

6.2 Hạn chế của hệ thống

Mặc dù HybridRAG đã mang lại kết quả tốt hơn so với từng phương pháp riêng lẻ nhưng hệ thống vẫn còn một số hạn chế nhất định:

Hệ thống được triển khai đơn giản mang tính tĩnh để phục vụ cho việc nghiên cứu môn học. Nền dữ liệu trong vector database và Knowledge Graph được xây dựng tại một thời điểm cố định và chưa có cơ chế tự động cập nhật khi xuất hiện dữ liệu hoặc tài liệu mới, trong khi thông tin trong thực tế luôn thay đổi và cần được cập nhật thường xuyên.

Cách kết hợp thông tin giữa VectorRAG và GraphRAG còn đơn giản. Việc ghép ngữ cảnh từ hai nguồn theo một thứ tự cố định có thể chưa phản ánh đúng mức độ quan trọng của từng nguồn thông tin đối với từng loại truy vấn.

6.3 Hướng nghiên cứu mở rộng

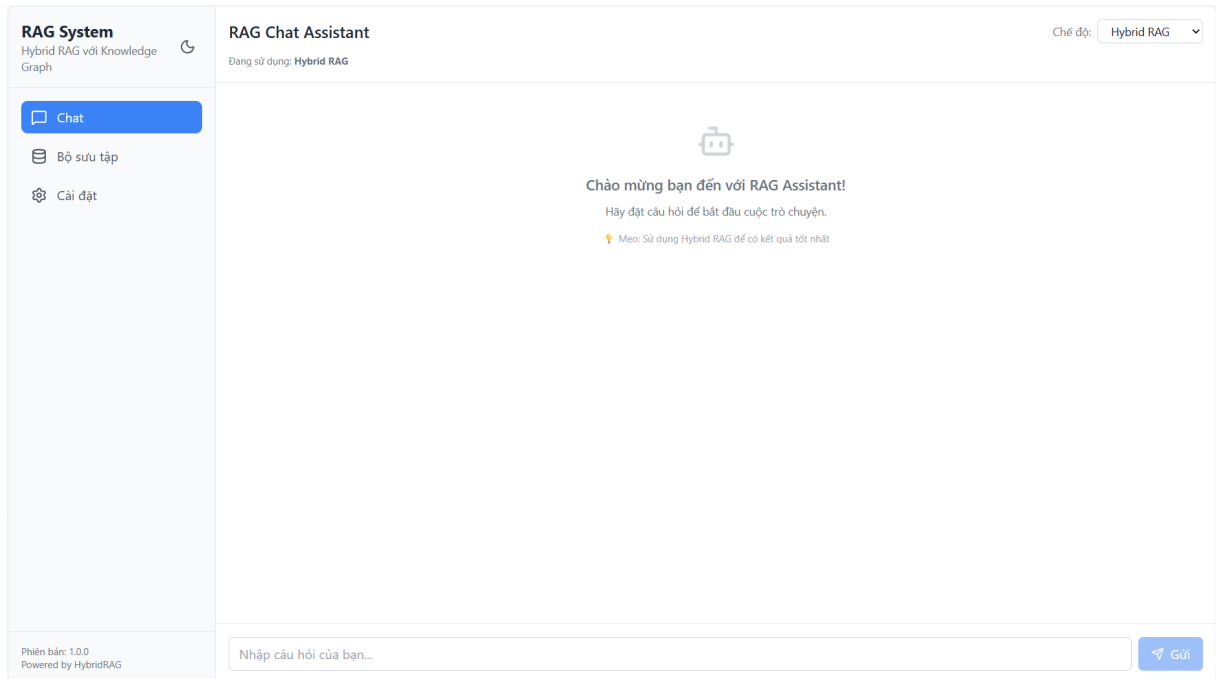
Để khắc phục các hạn chế trên và tiếp tục phát triển hệ thống theo hướng hoàn thiện hơn, một số hướng có thể áp dụng cho các nghiên cứu tương lai:

- Cập nhật dữ liệu tự động: Xây dựng cơ chế cho phép hệ thống tự động thêm, bớt và cập nhật thông tin mới khi dữ liệu thay đổi, nhằm đảm bảo hệ thống luôn sử dụng nguồn tri thức mới nhất.
- Làm sạch dữ liệu: Hiện tại, dữ liệu trong đồ thị có tồn tại trường hợp một thực thể có thể được biểu diễn bởi nhiều nút. Cách xử lý hiện tại mà nhóm áp dụng có thể lấy được toàn bộ các nút này, nhưng về lâu dài, để tối ưu tốc độ xử lý và không gian lưu trữ, cần thực hiện và đồng bộ các nút trùng lặp.
- Thử nghiệm trên nhiều lĩnh vực khác: Áp dụng HybridRAG cho các miền dữ liệu lớn hơn, cho các nhiệm vụ phức tạp, đòi hỏi độ chính xác cao.

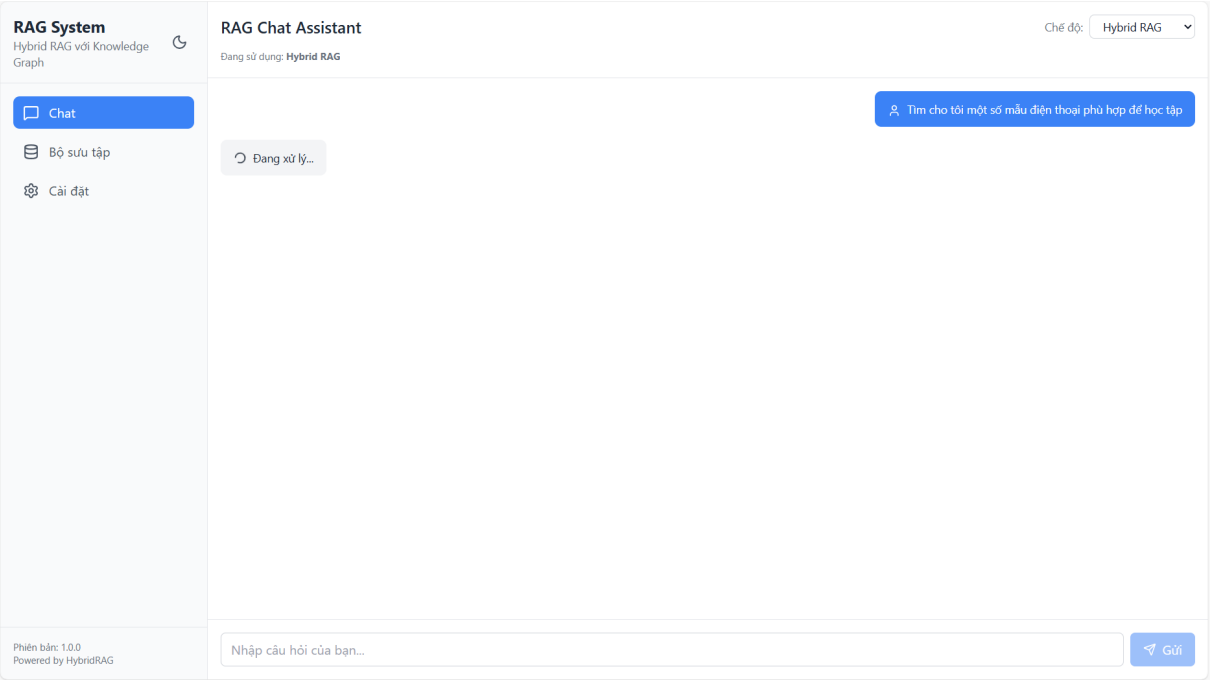
Chương 7

Hệ thống hỏi đáp

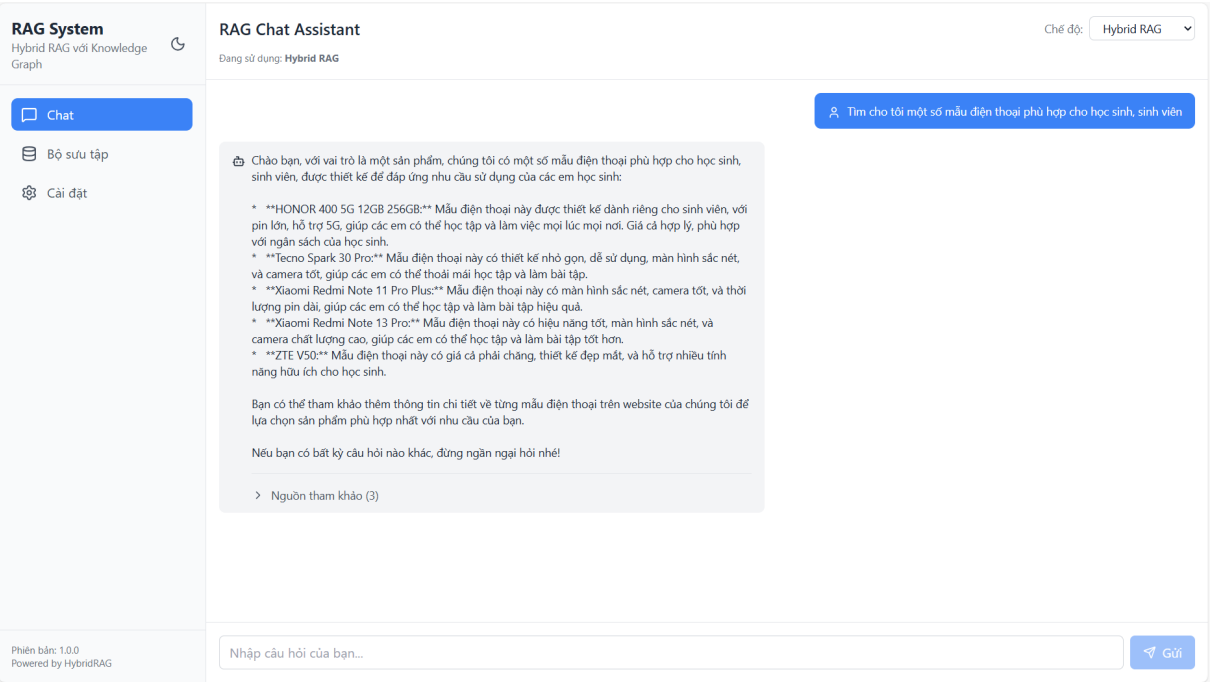
7.1 Một số hình ảnh chương trình



Hình 7.1: Giao diện ban đầu của chương trình



Hình 7.2: Giao diện chương trình khi đang xử lý truy vấn



Hình 7.3: Giao diện chương trình hiển thị kết quả trả lời

7.2 Ví dụ câu hỏi – câu trả lời

7.2.1 Ví dụ 1: Truy vấn gợi ý sản phẩm

Câu hỏi: Cho tôi danh sách điện thoại chất lượng tốt cho sinh viên.

Câu trả lời của hệ thống:

Chào bạn, rất vui được hỗ trợ bạn tìm kiếm điện thoại chất lượng tốt cho sinh viên! Dựa trên các thông tin bạn cung cấp, chúng tôi có một số gợi ý sau đây:

1. HONOR 400 5G 12GB 256GB: Đây là một lựa chọn rất tốt, giá cả phải chăng, phù hợp với sinh viên học tập và sinh viên mới tốt nghiệp. Giá bán hiện tại là 11.990.000đ.
2. Realme Note 60: Mẫu điện thoại này cũng được đánh giá cao về chất lượng và hiệu năng, đặc biệt là dành cho sinh viên. Giá bán là 1.499.000đ.
3. OPPO A5i 4GB 128GB: Nếu bạn muốn một chiếc điện thoại có thiết kế đẹp và giá cả phải chăng, OPPO A5i 4GB 128GB là một lựa chọn đáng cân nhắc. Giá bán là 2.499.000đ.

Bạn có muốn tìm hiểu thêm về bất kỳ mẫu điện thoại nào không? Hoặc, nếu bạn có bất kỳ yêu cầu cụ thể nào (ví dụ: ngân sách, tính năng đặc biệt), tôi sẽ giúp bạn tìm được chiếc điện thoại ưng ý nhất nhé!

7.2.2 Ví dụ 2: So sánh sản phẩm

Câu hỏi: So sánh iPhone 15 Plus và iPhone 15 Pro Max

Câu trả lời của hệ thống:

Chào bạn, so sánh iPhone 15 Plus và iPhone 15 Pro Max, đây là những điểm khác biệt chính:

- Hiệu năng: iPhone 15 Pro Max có chip A17 Bionic mạnh mẽ hơn, giúp xử lý tốt hơn các tác vụ nặng, đặc biệt là game và chỉnh sửa video.
- Camera: iPhone 15 Pro Max có hệ thống camera sau tốt hơn, đặc biệt là khả năng chụp ảnh trong điều kiện ánh sáng yếu.
- Màn hình: iPhone 15 Pro Max có màn hình Dynamic Island, kích thước lớn hơn và độ sáng cao hơn, mang lại trải nghiệm xem và sử dụng tốt hơn.

- Thiết kế: iPhone 15 Pro Max có thiết kế và vật liệu cao cấp hơn, với khung viền mỏng hơn và màu sắc khác biệt.
- Pin: iPhone 15 Pro Max có thể dùng lâu hơn so với iPhone 15 Plus.
- Giá: iPhone 15 Plus thường có giá rẻ hơn so với iPhone 15 Pro Max.

Tóm lại, nếu bạn ưu tiên hiệu năng, camera và màn hình, iPhone 15 Pro Max là lựa chọn tốt hơn. Nếu bạn cần một chiếc điện thoại tầm trung với giá cả phải chăng, iPhone 15 Plus là một lựa chọn hợp lý.

Bạn có câu hỏi nào khác không?

7.2.3 Ví dụ 3: Cung cấp thông tin

Câu hỏi: Hãy cho tôi biết các thông số kỹ thuật của Samsung Galaxy A56

Câu trả lời của hệ thống:

Chào bạn, sản phẩm Samsung Galaxy A56 5G có các thông số kỹ thuật sau:

- Pin: 5000mAh
- Chipset: Exynos 1580
- CPU: 8 nhân 2.9GHz, 2.6GHz, 1.9GHz
- Camera sau: 50.0 MP, F/1.8 + 12.0 MP, F/2.2 + 5.0 MP, F/2.4
- Loại CPU: 8 nhân 2.9GHz, 2.6GHz, 1.9GHz
- Camera trước: 12.0 MP, F/2.2
- Bộ nhớ trong: 128 GB
- RAM: 8 GB
- Hệ điều hành: Android
- Màn hình: 6.7 inches, 16 triệu màu, 120 Hz, 1200 nit, kính cường lực Corning Gorilla Glass Victus+
- Công nghệ NFC: Có
- Dung lượng RAM: 8 GB
- Tính năng màn hình: Super AMOLED

- Kích thước màn hình: 6.7 inches
- Độ phân giải màn hình: 1080 x 2340 pixels (FullHD+)
- Giới thiệu sản phẩm: Đặc điểm nổi bật của Samsung Galaxy A56 5G 8GB 128GB

Bạn có muốn biết thêm chi tiết nào không?

Tài liệu tham khảo

Tiếng Anh

- [1] Sarmah, Bhaskarjit et al. “HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction”. In: *arXiv preprint arXiv:2408.04948* (2024). arXiv: 2408.04948 [cs.CL].